

GUÍA TÉCNICA - PdfQuickDebug (Capacitación)

Herramienta de **debugging/desarrollo** para generar PDFs de **facturación electrónica colombiana (DIAN)** usando **QuestPDF**. Esta guía explica de dónde salen los datos, cómo funciona cada parte del proyecto y el **paso a paso para agregar un campo desde cero**.

1. ¿Qué hace el proyecto?

PdfQuickDebug es un ejecutable console (.NET 8) que:

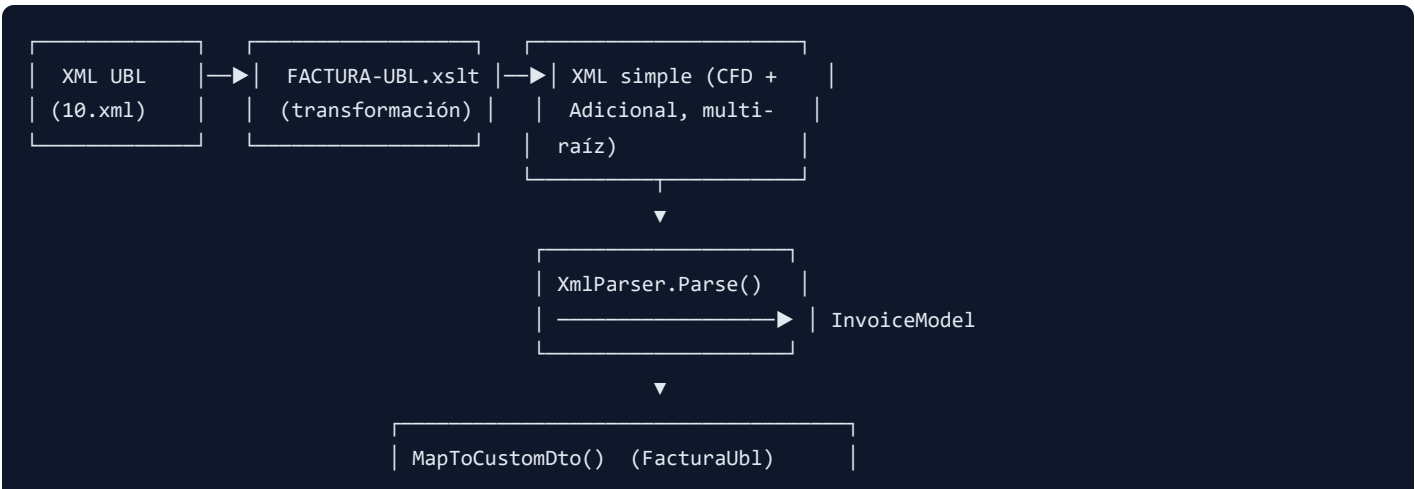
1. Lee un **XML UBL** de factura electrónica (la factura ya generada/validada).
2. Lo **transforma** con un XSLT (`.xslt`) a un "XML simple" de trabajo.
3. **Parsea** ese XML simple a un modelo en memoria (`InvoiceModel`).
4. Lo **mapea** a un DTO por cliente (`DtoUbl`), extrayendo campos personalizados.
5. **Renderiza un PDF** con QuestPDF (factura o nota crédito) y lo guarda en disco.

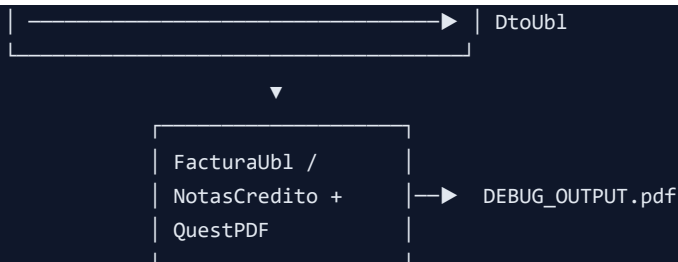
Además, genera automáticamente una copia transformada del template y DTO listos para copiarse al servicio real de generación de PDFs (`Services.PdfGenerator`).

Ubicación de carpetas

Carpeta	Contenido
DebugPDF/PdfQuickDebug/	Proyecto .NET (código fuente)
DebugPDF/PdfQuickDebug/Core/	Modelos, XSLT y parser
DebugPDF/PdfQuickDebug/Templates/	Plantillas de PDF y DTO
DebugPDF/PdfQuickDebug/Utilities/	Utilidades (números a letras, barras, QR helpers)
DataTest/	Archivos de prueba: XML, XSLT, PDF de salida, exportaciones

2. Flujo de datos (arquitectura)





Los 6 pasos del programa (Program.cs):

1. [1/5] Cargar XML y XSLT desde rutas configuradas al inicio del archivo.
2. [2/5] Aplicar XSLT (`XsltTransform.Transform`) y validar que existan `<CFD>` y `<Adicional>` .
3. [3/5] Parsear el XML transformado a `InvoiceModel` (`XmlParser.Parse`).
4. [4/5] Mapear a `DtoUbl` (`MapToCustomDto`) y generar el PDF (`GeneratePdf`).
5. [5/5] Guardar el PDF en `OUTPUT_PDF` .
6. [6/6] Exportar template + DTO al servicio (`ExportarTemplateParaServicio`).

3. ¿De dónde se toman los datos?

3.1 El XML UBL de entrada (DataTest/10.xml)

Es una **factura electrónica UBL 2.1** firmada, con estas zonas relevantes:

```

<Invoice xmlns="urn:...:Invoice-2" xmlns:cac="urn:...:CommonAggregateComponents-2"
  xmlns:cbc="urn:...:CommonBasicComponents-2" xmlns:ext="urn:...:CommonExtensionComponents-2"
  xmlns:sts="dian:gov:co:facturaelectronica:Structures-2-1" ...>
  
```

- `ext:UBLExtensions/ext:UBLExtension` + `sts:DianExtensions` : autorización DIAN, rango de numeración, QR URL, etc.
- `ds:Signature` : firma digital (de aquí sale `firma_digital`).
- `dif:CustomFieldExtension` : campos personalizados de la empresa emisora (fuente principal de "CustomFields"). Ejemplo real del archivo de prueba:

```

<CustomFieldExtension>
  <CustomField Name="CPDF" Value="factura-ubl"/>
  <CustomField Name="f_20_00007591_G502_1_ZONA" Value=""/>
  <CustomField Name="f_20_00007592_G502_1_RUTA" Value=""/>
  <CustomField Name="f_cargue" Value="MD261581"/>
  <CustomField Name="f_vehiculo_cargue" Value="WDP-236"/>
</CustomFieldExtension>
  
```

- `cbc:ID` (ej. `SETT1902408`) → número de documento; `cbc:UUID` → CUFE.
- `cbc:IssueDate` / `cbc:IssueTime` / `cbc:DueDate` → fechas.
- `cbc:Note` → notas / comentario del documento.
- `cac:OrderReference` → pedido: `cbc:ID` , `cbc:SalesOrderID` (→ `NumPedido`), `cbc:IssueDate` .
- `cac:AccountingSupplierParty` → emisor (compañía, dirección, NIT `CompanyID` , contacto).
- `cac:AccountingCustomerParty` → cliente/receptor (NIT, nombre, dirección, teléfono, correo).
- `cac:PaymentMeans` → medio de pago (`cbc:PaymentMeansCode`) y fecha de vencimiento.
- `cac:LegalMonetaryTotal` → totales (`LineExtensionAmount` , `PayableAmount` , ...).
- `cac:InvoiceLine` → cada ítem:

- `cbc:ID` (línea), `cbc:InvoicedQuantity` (cantidad/pacas),
- `cac:Item/cbc:Description`, `SellersItemIdentification/cbc:ID` (tiquete/código),
- `StandardItemIdentification/cbc:ID` (código de barras),
- `cac:Price/cbc:PriceAmount` (precio).

3.2 El XSLT (`DataTest/FACTURA-UBL.xslt`)

Es el **punto entre el XML UBL y el modelo**. Declara variables globales que "apuntan" hacia zonas del XML UBL:

```
<xsl:variable name="InfoAdicional"
  select="/fe:Invoice/ext:UBLExtensions/ext:UBLExtension/ext:ExtensionContent/dif:CustomFieldExtension/dif:CustomFi
```

Y produce un **XML simple con 2 raíces**:

- **<CFD>** → con los sub-elementos que el parser entiende: `<Folio>`, `<Documento>`, `<Emisor>`, `<Receptor>`, `<informacion_adicional>`, `<Detalle>` (uno por línea), `<QR>`, etc. Cada dato se expone como **atributo**: por ejemplo `<Documento numero_cd="1902408" total_am="34154.00" .../>`. El `Detalle` expone atributos como `cantidad_nu`, `cantidad_unit`, `descripcion_tx`, `tventa`, `unidad_medida_cd`, `item_subtotal_am`, `item_total_am`, `lote`, `bodega`, etc.
- **<Adicional>** → con la información "extra" del cliente:
- Atributos como `forma_pago_cd`, `notas_tx`.
- Un `<Campo clave="..." valor="...">` por cada `CustomField` del XML (`$InfoAdicional`).
- Campos "fijos" que el template necesita: `LogoBase64` (logo en base64), `ResolucionTexto`, `ValorLetras`.

```
<Adicional forma_pago_cd="..." notas_tx="...">
  <xsl:for-each select="$InfoAdicional">
    <Campo clave="{@Name}" valor="{@Value}" />
  </xsl:for-each>
  <Campo clave="LogoBase64">
    <xsl:attribute name="valor">
      <xsl:text>data:image/png;base64,</xsl:text>
      <xsl:value-of select="$LOGO_BASE64" />
    </xsl:attribute>
  </Campo>
  ...
</Adicional>
```

Clave del flujo: el parser lee **atributos por nombre exacto**. El programa de XSLT en `DataTest` y el código en `Core/XmlParser.cs` y `Templates/*.cs` deben usar **el mismo nombre de atributo** para que el dato llegue al PDF.

4. Carpeta `Core/`

4.1 `Models.cs` — Los modelos

Define la estructura de datos en memoria. Todo lo que pueda mostrar el PDF vive aquí (o en el DTO que hereda de esto).

- `InvoiceModel` — raíz del modelo:

- **Folio** (datos de autorización DIAN), **Document** (número, fechas, totales, CUFE/CUDE, firma), **Issuer / Customer** (PartyInfo), **Lines** (List), **Totals**, **QR**, **Additional**, **InformacionAdditional**.
- **PartyInfo** — emisor o cliente: **TaxId** (NIT), **Name**, **Name_Comercial**, **Address**, **City**, **Phone**, **Email**, **TaxScheme**.
- **DocumentInfo** — **Number**, **Prefix**, **IssueDate**, **HoraEmision**, **DueDate**, **Currency**, **Total_am**, **Total_ibua**, **ValorBruto**, **TotalIVA**, **TotalLetras**, **Comentario_ds**, **FacturaBase**, **CUFE**, **CUDE**, **FirmaDigital**, etc.
- **LineItemInfo** — cada fila de producto: **LineNumber**, **Description**, **Tiquete**, **Codigo_cd**, **TVenta**, **Cant_Und**, **Quantity**, **Unit_Meditation**, **UnitPrice**, **PrecioAntesIVA**, **ValorDescuento**, **TaxRate**, **TaxValor**, **LineTotal**, **Ibua**.
- **AdditionalInfo** — **PaymentMethod**, **PaymentCondition**, **Notes** y **CustomFields** (Dictionary<string,string>) que es donde aterrizan todos los <Campo> del **Adicional**.
- **AdditionalInformation** — **Telefono_Receptor**, **NumCargue**, **OC**, **NumPedido**, **Zona**.

Además define:

- **IInvoiceTemplate** — contrato que todo template debe implementar:
- `byte[] GeneratePdf(InvoiceModel model)` → genera el PDF.
- `TemplateMetadata GetMetadata()` → datos del template (NIT, nombre cliente, versión).
- `InvoiceModel MapToCustomDto(InvoiceModel baseModel)` → transforma el modelo genérico en el DTO personalizado del cliente (por defecto devuelve el mismo modelo).
- **TemplateMetadata** — **Nit**, **TemplateName**, **ClientName**, **Version**, **LastUpdated**.

4.2 XsltTransform.cs — Transformación XSLT

- `Transform(xmlContent, xsltContent)` :
- Compila el XSLT con `XslCompiledTransform` (con función `document()` habilitada).
- Escribe la salida con `ConformanceLevel.Fragment` **porque el XSLT produce 2 raíces** (CFD y Adicional).
- `ValidateTransformedXml(transformedXml)` :
- Envuelve el resultado en <Root> y verifica que exista <CFD> y <Adicional>, y cuenta cuántos <Campo> (CustomFields) trae.

4.3 XmlParser.cs — Parser

Convierte el XML simple en `InvoiceModel` :

- `Parse(transformedXml)` :
- Envuelve el XML en <Root> (para soportar múltiples raíces).
- Busca el elemento <CFD> (obligatorio) y <Adicional> (opcional).
- Llama a los métodos `Parse*` por sección: **Folio**, **Documento**, **Emisor**, **Receptor**, **Detalle** (todas las líneas), **Totales**, **QR**, **Adicional**, **informacion_adicional**.
- **ParseLine** lee los atributos del <Detalle> y arma `LineItemInfo`.
- **Helpers importantes:**
- `ParseDecimal(...)` : quita \$, comas y espacios antes de convertir.
- `ParseDate(...)` : espera formato yyyy-MM-dd ; si no coincide devuelve `DateTime.MinValue`.
- `ParseInt(...)` : equivalente para enteros.
- **ParseAdditional** llena `AdditionalInfo`, incluyendo `CustomFields` como diccionario **clave → valor** desde los elementos <Campo>.

Nota: `ParseLine` tiene un `Console.WriteLine` de debug en el parseo de `cantidad_unit`. No es un error, pero ensucia la consola; si molesta se puede eliminar (Core/XmlParser.cs, el `ParseLine`).

5. Carpeta `Templates/`

5.1 DtoUbl.cs — ¿Para qué sirve el DTO?

`DtoUbl` hereda de `InvoiceModel` y agrega **propiedades tipadas** para los campos personalizados de AJECOLOMBIA. En vez de andar leyendo `model.Additional.CustomFields["f_..."]` (texto libre, propenso a errores de tipeo), el template usa `dto.Zona`, `dto.Ruta`, etc.

Propiedades que agrega:

```
public class DtoUbl : InvoiceModel
{
    public string Zona { get; set; } = string.Empty;
    public string Ruta { get; set; } = string.Empty;
    public string PlacaVehiculo { get; set; } = string.Empty;
    public string NumCargue { get; set; } = string.Empty;
    public string OrdenCompra { get; set; } = string.Empty;
    public string NumPedido { get; set; } = string.Empty;
    public string HoraGeneracion { get; set; } = string.Empty;
    public string ResolucionTexto { get; set; } = string.Empty;
    public string LogoBase64 { get; set; } = string.Empty;
    public string ValorLetras { get; set; } = string.Empty;
}
```

Flujo de un campo personalizado:

1. El XML trae `<CustomField Name="f_vehiculo_cargue" Value="WDP-236"/>`.
2. El XSLT lo convierte en `<Campo clave="f_vehiculo_cargue" valor="WDP-236"/>` dentro de `<Adicional>`.
3. El parser lo mete al diccionario `model.Additional.CustomFields["f_vehiculo_cargue"]`.
4. `MapToCustomDto` lo copia a la propiedad tipada `dto.PlacaVehiculo`.
5. El template hace `dto.PlacaVehiculo` para imprimirlo.

5.2 FacturaUbl.cs — Template de factura

- Implementa `IInvoiceTemplate`. Página tamaño **612 × 391 pt** (formato térmico/rollo).
- **Paginación:** divide las líneas en grupos de **12 por página**; por cada página crea un `container.Page`.
- **Estructura de cada página:**
 - `Header` → `ComposeHeader` (logo + datos emisor + factura/QR) y `ComposeClientInfo` (datos del cliente en 3 columnas: nombre, zona/ruta, medio de pago...).
 - `Content` → `ComposeProductTable` (tabla de productos con 13 columnas).
 - `Footer` → `ComposeTotalsTable` (totales), "VALOR EN LETRAS", y `ComposeFooter` (texto legal, CUFE, firma digital, número de página).
- `MapToCustomDto` es el corazón del mapeo. Ejemplos de claves que usa:

```
Zona          = baseModel.InformacionAdditional?.Zona ?? GetCustomFieldStatic(baseModel, "f_20_00007591_G502_1_ZONA")
Ruta          = GetCustomFieldStatic(baseModel, "f_20_00007592_G502_1_RUTA"),
PlacaVehiculo = GetCustomFieldStatic(baseModel, "f_vehiculo_cargue"),
NumCargue     = baseModel.InformacionAdditional?.NumCargue ?? GetCustomFieldStatic(baseModel, "f_cargue"),
```

• Helpers internos:

- `GetCustomField(model, key, default)` → busca en `CustomFields` con fallback.
- `SafeGenerateQRCode(url)` → genera el QR con `QRCode` (si falla, devuelve `null`).
- `SafeLoadLogo(logo)` → soporta: ruta local, URL HTTP/HTTPS y `data:image/...;base64,...`.
- `CreaListaPaginadaCant_Und(...)` → agrega cantidades unidad por página.

- **Últimos cambios aplicados (para referencia):**
- La columna **TIPO VENTA** muestra "Bonifi" cuando `PrecioAntesIVA == 0`, si no muestra `TVenta`.
- Las **cantidades se intercambiaron**: ahora **CANT UND** muestra `Quantity` (cantidad/pacas) y **CANT PACAS** muestra `Cant_Und`.

Al final del archivo están las clases auxiliares `BrandColors` (paleta AJE) e `InvoiceTableStyles` (bordes, fuentes, paddings de la tabla de ítems).

5.3 `NotasCredito.cs` — Template de nota crédito

- Muy parecido a `FacturaUbl`, pero:
- Página **612 × 399 pt**.
- Encabezado dice "NOTA CREDITO DE LA FACTURA ELECTRONICA DE VENTA".
- El `Footer` muestra **CUDE** (además de CUFE).
- `GetMetadata()` devuelve NIT `800153993`.
- `MapToCustomDto` usa claves distintas en algunos campos (p. ej. "Zona", "Ruta", "Placa"), algo que **hay que homogeneizar** (ver sección 8).

6. `Utilities/PDF_Utilities.cs`

Función	Qué hace
<code>PDF_Utils.NumeroALetrasCOP(decimal)</code>	Convierte un número en "DOS MIL TRESCIENTOS CUARENTA Y CINCO PESOS".
<code>SkiaSharpHelpers.SkiaSharpCanvas</code>	Permite dibujar con SkiaSharp dentro de un contenedor de QuestPDF (SVG bridge).
<code>DrawBarCodeExtensions.DrawBarCode</code>	Dibuja un código de barras (ZXing, Code128 por defecto).
<code>StringsProcessing.GenerateEan13(string)</code>	Calcula y agrega el dígito de control a 12 dígitos (EAN-13).

7. `Program.cs` — Configuración y orquestación

7.1 Zona de configuración (solo editar esta sección)

```
const string XML_PATH      = @"...\DataTest\10.xml";
const string XSLT_PATH     = @"...\DataTest\FACTURA-UBL.xslt";
const string OUTPUT_PDF   = @"...\DataTest\DEBUG_OUTPUT.pdf";
const string NIT           = "800153993";    // → carpeta de exportación _800153993/

const bool EXPORTAR_TEMPLATE = true;         // exporta template con namespaces correctos
const string EXPORT_BASE_PATH = @"...\DataTest";
const string SERVICE_TEMPLATE_PATH = @"E:\DV-SCM\..."; // (sin uso actual)

const bool ENABLE_COMPANION = true;          // QuestPDF Companion en http://localhost:12500
const int COMPANION_PORT    = 12500;
```

7.2 Cómo se genera el PDF (pasos 1-5)

1. Valida que existan el XML y el XSLT (sino, imprime error y corta).

2. Carga ambos textos; transforma XML con XSLT; valida estructura (`ValidateTransformedXml`).
3. Parse con `XmlParser.Parse`, imprime resumen (documento, cliente, total, líneas, CustomFields).
4. `var customModel = template.MapToCustomDto(model)` (aplica FacturaUbl, pero NotasCredito también se instancia).
5. `var pdfBytes = template.GeneratePdf(customModel)` → guarda en `OUTPUT_PDF` . - Para preview en navegador (hot reload) existe `PreviewOnQuestPDFCompanion(...)` (comentado: líneas 146-147 de Program.cs).

7.3 Exportación al servicio (paso 6)

`ExportarTemplateParaServicio()` :

- Crea `EXPORT_BASE_PATH/_800153993/` (con el NIT configurado).
- Copia `FacturaUbl.cs`, `NotasCredito.cs` y `DtoUbl.cs` desde `Templates/` :
- Reemplaza `using PdfQuickDebug.Core;` → `using Services.PdfGenerator.Domain.Models;` .
- Reemplaza `namespace PdfQuickDebug.Templates;` → `namespace Services.PdfGenerator.Infrastructure.Templates._800153993;` .
- El resultado se copia al servicio real en `Services.PdfGenerator/Infrastructure/Templates/_800153993/` .

8. PASO A PASO: Cómo agregar un campo desde cero

★ La regla de oro

Un campo nuevo recorre siempre la misma cadena:

XML UBL → atributo en XSLT → propiedad en el modelo/DTO → render en el template

El dato **nace** en el XML UBL (con o sin `CustomField`). El **XSLT** lo expone. El **parser** (o `MapToCustomDto`) lo lleva a una propiedad. El **template** lo imprime.

Hay dos rutas según el origen:

Ruta	Origen del dato	Pasos necesarios
A. CustomField (campos personalizados)	<code><CustomField Name="..." Value="..."></code> en el XML	XSLT ya genera <code><Campo></code> → el parser no se toca ; solo DTO + <code>MapToCustomDto</code> + template
B. Grupo de línea (por producto)	elementos dentro de <code><cac:InvoiceLine></code>	XSLT (<code>Detalle</code>) + parser (<code>ParseLine</code>) + modelo (<code>LineItemInfo</code>) + template

Ejemplo A: Agregar a cabecera un CustomField (campo de factura)

Queremos mostrar "**Nombre del vendedor**" en el bloque de cliente de la factura.

Paso 1 — Verificar el dato en el XML UBL

Busca el campo en la sección `CustomFieldExtension` :

```
<CustomField Name="vendedornombre" Value="JUAN PEREZ"/>
```

Si el dato no viene como `CustomField`, la ruta es distinta (tendrías que exponerlo en el XSLT, ver sección "Exponer dato que NO es CustomField" más abajo).

Paso 2 — Confirmar que el XSLT lo convierta a `<Campo>`

El XSLT ya hace esto por todos los CustomFields de forma automática:

```
<xsl:for-each select="$InfoAdicional">
  <Campo clave="{@Name}" valor="{@Value}"/>
</xsl:for-each>
```

Resultado intermedio (XML simple):

```
<Adicional ...>
  ...
  <Campo clave="vendedornombre" valor="JUAN PEREZ"/>
```

Conclusión: no hay que tocar el XSLT para CustomFields (el parser los guarda todos en `model.Additional.CustomFields`).

Paso 3 — Agregar la propiedad al DTO (`Templates/DtoUbl.cs`)

```
/// <summary>Nombre del vendedor</summary>
public string Vendedor { get; set; } = string.Empty;
```

Paso 4 — Mapéalo en `MapToCustomDto` de `Templates/FacturaUbl.cs`

```
// (dentro del new DtoUbl { ... })
Vendedor = GetCustomFieldStatic(baseModel, "vendedornombre"),
```

Con esto ya llega tipeado: `dto.Vendedor == "JUAN PEREZ"`.

Paso 5 — Renderízalo en el template (`FacturaUbl.cs`)

Dentro de `ComposeClientInfo`, por ejemplo, agregar al bloque de etiquetas y de valores:

```
// Columna de etiquetas
col.Item().Text("VENDEDOR: ").FontSize(6f).Bold();

// Columna de valores
col.Item().Text(dto?.Vendedor ?? "").FontSize(6f);
```

Paso 6 — Ejecutar y verificar

```
dotnet run
```

Descartar la UI y ver el PDF: `DataTest/DEBUG_OUTPUT.pdf`. (O poner `ENABLE_COMPANION` /descomentar `PreviewOnQuestPDFCompanion` para preview en vivo.)

Paso 7 — (Opcional) Exportar al servicio

Con `EXPORTAR_TEMPLATE = true` , los cambios se reflejan en el archivo exportado (`DataTest/_800153993/FacturaUbl.cs`) con el namespace y `using` correctos.

Ejemplo B: Agregar un campo a nivel de línea (por producto)

Queremos mostrar el **lote** de cada producto como una columna nueva en la tabla.

Paso 1 — Dato en el XML UBL

Dentro de cada `InvoiceLine` :

```
<cac:InvoiceLine>
  ...
  <cac:Item>
    <cac:ItemInstance>
      <cac:LotIdentification>
        <cbc:LotNumberID>LOTE-001</cbc:LotNumberID>
      </cac:LotIdentification>
    </cac:ItemInstance>
  </cac:Item>
</cac:InvoiceLine>
```

Paso 2 — Exponerlo en el XSLT (`FACTURA-UBL.xslt`)

En el `<Detalle>` agregar (o verificar) el atributo:

```
<Detalle linea_nu="{position()}"
  ...
  lote="{cac:Item/cac:ItemInstance/cac:LotIdentification/cbc:LotNumberID}"
  ...>
```

Ojo: si NO es un `CustomField` , **sí hay que tocar el XSLT**, porque el parser solo ve lo que el XSLT expone como atributo.

Paso 3 — Agregar la propiedad al modelo (`Core/Models.cs`)

```
public class LineItemInfo
{
  // ... propiedades existentes
  public string Lote { get; set; } = string.Empty;
}
```

Paso 4 — Parsearlo (`Core/XmlParser.cs` , método `ParseLine`)

```
Lote = elem.Attribute("lote")?.Value ?? string.Empty,
```

Paso 5 — Renderizarlo en la tabla (`FacturaUbl.cs` , `ComposeProductTable`)

5a. Agregar (o ajustar) la definición de columnas:

```
table.ColumnsDefinition(columns =>
{
    columns.RelativeColumn(4.8f);    // DESCRIPCION
    columns.ConstantColumn(22);      // U M
    columns.ConstantColumn(44);      // CODIGO PRODUCTO
    columns.ConstantColumn(44);      // COD. BARRAS
    columns.ConstantColumn(36);      // TIPO VENTA
    columns.ConstantColumn(30);      // CANT UND
    columns.ConstantColumn(33);      // CANT PACAS
    columns.ConstantColumn(53);      // PRECIO ANTES IVA
    columns.ConstantColumn(38);      // DCTO
    columns.ConstantColumn(27);      // IVA%
    columns.ConstantColumn(44);      // IVA
    columns.ConstantColumn(27);      // IBUA
    columns.ConstantColumn(40);      // LOTE (NUEVO)
    columns.ConstantColumn(50);      // VALOR TOTAL
});
```

5b. Agregar el encabezado en `table.Header(...)` :

```
HeaderCell("LOTE");
```

5c. Agregar la celda de datos en el `foreach (var line in lines)` :

```
table.Cell().Border(...).PaddingVertical(0.75f).PaddingHorizontal(1.1f)
    .AlignCenter().AlignMiddle()
    .Text(line.Lote.Trim() ?? string.Empty)
    .FontFamily(...).FontSize(...).FontColor(...);
```

5d. Ajustar la fila de "relleno" inferior que copia las divisiones de columna (el `column.Item().ExtendVertical().Row(...)` del final de `ComposeProductTable`), agregando un `row.ConstantItem(40).BorderRight(0.5f).BorderHorizontal(0.5f);` en la posición correcta.

Al cambiar el ancho de las columnas, revisa que la **tabla de totales** y los datos sigan alineados; si no, ajusta los anchos de `ComposeTotalsTable` .

Paso 6 — Verificar (igual que Ejemplo A, paso 6)

Exponer un dato que NO es CustomField (cabecera)

Si el dato no viene en `CustomFieldExtension` , hay que exponerlo manualmente en el XSLT. Dos opciones comunes:

Opción 1: atributo en `<informacion_adicional>` (si el dato es de cabecera):

```
<informacion_adicional ... Vendedor="{ $InfoAdicional[@Name = 'vendedornombre']/@Value}" />
```

Y luego leerlo en el parser (`ParseInformacionAdittional`) agregando la propiedad.

Opción 2: atributo en el elemento correspondiente (p. ej. `<Documento>`):

```
<Documento ... comentario_ds="{fe:Invoice/cbc:Note}" .../>
```

Después leerlo en `ParseDocument` .

9. Buenas prácticas y errores comunes

1. **Los nombres de atributo deben coincidir EXACTAMENTE** entre XSLT, parser y template. Un espacio o un tipeo hace que el dato llegue vacío al PDF.
2. **CustomFields te ahorra tocar el parser.** Si el campo sale por `CustomFieldExtension` , el XSLT ya lo vuelve `<Campo>` y el parser ya lo guarda en `CustomFields` . Solo necesitas DTO + `MapToCustomDto` + template.
3. **Claves de CustomFields inconsistentes entre templates.** `FacturaUbl` usa `f_20_00007591_G502_1_ZONA` / `f_vehiculo_cargue` / `f_cargue` , mientras que `NotasCredito` usa `Zona` / `Ruta` / `Placa` / `NumCargue` . Si el XML cambia de nombres, un template sí muestra el dato y el otro no. **Homogeneízalas.**
4. **NITs inconsistentes.** `Program.cs` exporta con NIT `800153993` ; `FacturaUbl.GetMetadata()` devuelve `830081407` ; el comentario del DTO menciona `1193122070` . Decide cuál es el NIT oficial del cliente y úsalo en todos lados.
5. **DateTimeOffset.Parse puede crashear.** En `NotasCredito.cs` se usa `DateTimeOffset.Parse(model.Document?.HoraEmision)` directo; si `HoraEmision` viene vacío o mal formateado, explota. `FacturaUbl` usa `TryParse` (correcto). Usa siempre `TryParse` .
6. **Encoding.** Se registra `CodePagesEncodingProvider` para el fix Windows-1252/UTF-8, y se carga la fuente `Liberation Sans` en `Program.cs`. No borres estas líneas.
7. **No cambiar la licencia de QuestPDF.** Está en `LicenseType.Community` ; QuestPDF exige licencia comercial a partir de cierto umbral. No la "silencies" saltándote el chequeo.
8. **La paginación usa 12 hardcodeado** (pese a existir `itemsPorPagina = 12`). Si cambias el ítems por página, cámbialo en el `GroupBy(x => x.index / 12)` .
9. **Hay código muerto** (`ListaPrueba` , `CreaListaPaginadaCant_Und` , `SERVICE_TEMPLATE_PATH` , `SafeFixEncodingIssues` , `totalUnidades / totalQuantity` en `NotasCredito`). No asustarse, pero evita acumular más.
10. **PaymentCondition nunca se llena** en el parser, por lo que "CONDICION DE PAGO" aparece vacío. Si lo necesitas, expón el dato en el XSLT y parsearlo.

10. Cómo añadir un campo de un tag del XML al template FacturaUbl

Este es el procedimiento detallado y recomendaciones para exponer **cualquier tag del XML UBL** en el template `FacturaUbl.cs` (aplicable igual a `NotasCredito.cs`).

Regla del proyecto: para añadir o ajustar campos **no se toca** `Core\XmlParser.cs` , `Core\Models.cs` ni la transformación. Solo se editan `Templates\DtoUbl.cs` , `DataTest\FACTURA-UBL.xslt` y `Templates\FacturaUbl.cs` .

10.1 Recorrido de los datos (regla de oro)

```
XML UBL (NUEVO_XML.xml)
| FACTURA-UBL.xslt ← se toca SOLO si el tag NO es CustomField
▼
XML simple (atributos + <Campo clave="..." valor="..."/>)
| XmlParser.cs (CAJA NEGRA: no se toca; guarda TODO <Campo> en Additional.CustomFields)
▼
Modelo base (Models.cs: Document / Customer / Lines / Additional) (no se toca)
| MapToCustomDto() (Templates/FacturaUbl.cs) ← se toca (mapeo + GetCustomFieldStatic)
▼
DtoUbl.cs (propiedades tipadas + diccionarios) ← se toca SOLO si quieres el dato tipeado
| render (Compose..., celdas del PDF)
```

Para saber el origen de un campo que ya ves en el PDF (trazabilidad inversa), sigue la ruta opuesta: template → propiedad/diccionario → XmlParser.cs → atributo/ <Campo> en XSLT → nodo del XML. Ejemplo real: NombreClienteLabel usa Customer.Name_Comercial (XmlParser.cs:95 , atributo nombre_comercial_ds) → FACTURA-UBL.xslt:491 (cac:AccountingCustomerParty/cac:Party/cac:Contact/cbc:Name) → NUEVO_XML.xml:312 .

10.2 Pregunta que decide la ruta

¿El dato vive en la sección <CustomFieldExtension><CustomField Name="..." Value="..." /> ?

- **SÍ → Ruta A (rápida):** no se toca el XSLT ni el parser (ver 10.3).
- **NO** (es un elemento cac: / cbc: como <cac:Contact><cbc:Name>) → **Ruta B:** hay que exponerlo manualmente en el XSLT (ver 10.4).

10.3 Ruta A — El tag es un CustomField

1. **Verificar el dato en** NUEVO_XML.xml , dentro de CustomFieldExtension :

```
xml <CustomField Name="f_direccion2_env" Value="CA 11A 1B73" />
```

2. **No tocar el XSLT.** El XSLT ya convierte **todos** los CustomFields en <Campo> (\$InfoAdicional , FACTURA-UBL.xslt:38-40) y el parser los guarda en model.Additional.CustomFields .

3. **Agregar propiedad tipada** en Templates\DtoUbl.cs :

```
csharp /// <summary>Dirección 2 de envío (CustomField f_direccion2_env).</summary> public string Direccion2Env {
get; set; } = string.Empty;
```

4. **Mapear en MapToCustomDto** (Templates\FacturaUbl.cs , dentro del new DtoUbl { ... }):

```
csharp Direccion2Env = GetCustomFieldStatic(baseModel, "f_direccion2_env"),
```

GetCustomFieldStatic busca la clave en Additional.CustomFields con manejo seguro de nulos. Es la forma recomendada de leer CustomFields.

5. **Renderizar** en el bloque adecuado (etiqueta + valor):

```
csharp col.Item().Text("DIRECCION DESTINO: ").FontSize(6f).Bold(); col.Item().Text(dto?.Direccion2Env ??
"" ).FontSize(6f);
```

6. **Validar:** dotnet build (0 errores) + dotnet run , revisar DEBUG_OUTPUT.pdf .

Caso real de esta sesión: ConDireccionDestino (FacturaUbl.cs:1101) combina los CustomFields f_direccion1_env + f_direccion2_env con GetCustomFieldStatic .

10.4 Ruta B — El tag es un elemento cac: / cbc: (NO CustomField)

1. **Ubicar el nodo exacto en** NUEVO_XML.xml : <cac:Party>/<cac:PartyLegalEntity>/<cbc:RegistrationName>SANDOVAL CRISTANCHO ISAIAS</cbc:RegistrationName> .

2. **Exponerlo en el XSLT** (FACTURA-UBL.xslt) como <Campo ...> : así el parser lo guarda en Additional.CustomFields **sin tocar** Core\XmlParser.cs ni Core\Models.cs (el parser almacena automáticamente TODOS los <Campo clave="..."

```
valor="..."/>):
```

- **Cabecera (cliente / documento):** agregar un `<Campo>` nuevo en la sección de campos de cabecera, p. ej.:

```
xml <Campo clave="f_nombre_legal_cliente" valor="
{fe:Invoice/cac:AccountingCustomerParty/cac:Party/cac:PartyLegalEntity/cbc:RegistrationName}"/>
```

- **Por línea de producto:** emitir `<Campo clave="x_{position()}" valor="..."/>` dentro del `<xsl:for-each select="fe:Invoice/cac:InvoiceLine">` (patrón de `f_cod_barras_movto` en `FACTURA-UBL.xslt:996-1007`) y leerlo con `additional.CustomFields.GetValueOrDefault("x_{n}")`.
- 3. **Mapear en `MapToCustomDto`** (`Templates\FacturaUbl.cs`): leer con `GetCustomFieldStatic` (y agregar la propiedad tipada en `Templates\DtoUbl.cs` si quieres el dato tipeado):

```
csharp NombreLegalCliente = GetCustomFieldStatic(baseModel, "f_nombre_legal_cliente"),
```

Regla de oro: el `clave` del `<Campo>` del XSLT y la `"clave"` que pasas a `GetCustomFieldStatic` deben coincidir EXACTAMENTE.

4. **Renderizar** en el template y **validar** igual que la Ruta A.

5. **Validar** igual que la Ruta A.

10.5 Validación final (siempre)

```
dotnet build # exigir "0 Errores"
dotnet run # genera DataTest/DEBUG_OUTPUT.pdf y resuelve CustomFields
```

Si dudas del valor resuelto: agregar temporalmente `Console.WriteLine` en `MapToCustomDto`, correr, confirmar, **y luego quitarlo**. Al final confirmar que el export quedó sincronizado (`DataTest\_800153993\FacturaUbl.cs`) porque `EXPORTAR_TEMPLATE=true` lo reescribe en cada `dotnet run`.

10.6 Recomendaciones

1. **Regla de alcance:** solo `Templates\DtoUbl.cs`, `DataTest\FACTURA-UBL.xslt` y `Templates\FacturaUbl.cs`. Nunca tocar `Core\*` (parser/Models) ni la transformación.
2. **Nombres exactos** entre el `clave / valor` del `<Campo>` (XSLT), la clave en `GetCustomFieldStatic` y el DTO: un tipeo o un espacio hace que el dato llegue vacío al PDF.
3. **Mantén fallbacks** (p. ej. `Customer.Address` si el tag nuevo no viene) para no dejar celdas en blanco en XMLs antiguos.
4. **No rompas la agrupación por SKU:** `AgruparPorSku` agrupa por clave `Tiquete | f_motivo | um_base` (donde `um_base = Unit_Medition`, derivada de `f_um_base`). Si agregas campos por línea, recuerda que el grupo toma la primera línea como plantilla y suma cantidades/importes.
5. **Campo vacío vs. ausente:** usa `string.IsNullOrEmpty` y `TryGetValue / GetValueOrDefault` (nunca `Indexador["..."]` directo si la clave puede faltar).
6. **Valida siempre con el `NUEVO_XML.xml1` real** de la semana y revisa el PDF de salida.

11. Referencias de documentación existente

Archivo (en <code>DebugPDF/PdfQuickDebug/</code>)	Contenido
<code>COMO_DEBUGGEAR.md</code>	3 métodos de debugging (<code>Console.WriteLine</code> , C# Dev Kit, Visual Studio).
<code>CÓMO_USAR_EXPORTACION.md</code>	Cómo funciona la exportación automática al servicio.

Archivo (en <code>DebugPDF/PdfQuickDebug/</code>)	Contenido
<code>INSTALAR_EXTENSION.md</code>	Instalar extensión C# para breakpoints.
<code>DEBUGGING_SIN_EXTENSION.md</code>	Debugging con <code>Console.WriteLine</code> solamente.
<code>SOLUCION_DEFINITIVA.md</code> / <code>SOLUCION_ERROR_VSCODE.txt</code>	Solución al error "coreclr" de VS Code.

Resumen ejecutivo para el equipo nuevo:

- Los datos vienen del **XML UBL** y llegan al PDF pasando por **XSLT** → **parser** → **modelo/DTO** → **template**.
- El **DTO (DtoUbl)** es el "traductor" que convierte los CustomFields genéricos en propiedades tipadas fáciles de usar en el template.
- **Para agregar un campo:** sigue la regla de oro (XML → XSLT → modelo → render) o usa la vía corta de CustomFields (XSLT ya resuelto → DTO → `MapToCustomDto` → template).
- Al final siempre: `dotnet run` y revisar `DataTest/DEBUG_OUTPUT.pdf` .